experts.py — technical

`src/moe_pruning/models/olmoe/experts.py · 155 lines · moe_pruning.models.olmoe.experts`

Purpose

Fused-weight expert containers.

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`_swiglu_expert`

```
def _swiglu_expert(x: torch.Tensor, gate_up: torch.Tensor, down: torch.Tensor, act_fn) -> torch.Tensor
```

lines 24-26

1. `gate, up = (x @ gate_up.T).chunk(2)`; return `(act(gate) * up) @ down.T`.

`class FusedExpertView`

lines 29-39 · bases nn.Module

Callable view of one row of a fused expert container (owns no parameters).

`FusedExpertView.forward`

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

lines 37-39

Compute this expert's output from its row of the fused weights.

`class OlmoeReferenceExpert`

lines 42-52 · bases nn.Module

A single standalone original expert (used to measure reconstruction errors).

`OlmoeReferenceExpert.forward`

```
def forward(self, x: torch.Tensor) -> torch.Tensor
```

lines 51-52

Compute the expert's output (SwiGLU).

`class PrunedFusedExperts`

lines 55-155 · bases nn.Module

Drop-in replacement for `OlmoeExperts` with some experts removed.

`PrunedFusedExperts.__init__`

```
def __init__(self, num_experts: int, hidden_dim: int, intermediate_dim: int, act_fn, retained_ids:
list[int], device=None, dtype=None)
```

lines 64-85

1. Allocate `gate_up_proj [R, 2I, D]`, `down_proj [R, D, I]` (R = retained count), persistent buffer `retained_expert_ids`, empty `ModuleDict` `replaced`, and the mapping original id -> row.

`PrunedFusedExperts.compact`

```
def compact(cls, experts: nn.Module, remove: list[int]) -> 'PrunedFusedExperts'
```

lines 89-118

New container without the experts in `remove`.

1. Determine the retained ids before (all experts for an original container) and after removal.
2. `index_select` the rows of the remaining experts from the source tensors and clone them into new parameters (works on meta tensors too, used by the checkpoint loader).
3. Carry over existing replacement modules.

`PrunedFusedExperts.retained_ids`

```
def retained_ids(self) -> list[int]
```

lines 120-122

Original ids of the retained experts, in row order.

`PrunedFusedExperts.removed_ids`

```
def removed_ids(self) -> list[int]
```

lines 124-125

Original ids of removed experts.

`PrunedFusedExperts.expert`

```
def expert(self, expert_id: int) -> nn.Module
```

lines 127-133 · raises KeyError

Module for an expert id: a row view if retained, the replacement if removed.

1. Removed id -> its replacement module; retained id -> `FusedExpertView(self, row)`; unknown -> `KeyError`.

`PrunedFusedExperts.forward`

```
def forward(self, hidden_states: torch.Tensor, top_k_index: torch.Tensor, top_k_weights: torch.Tensor)
-> torch.Tensor
```

lines 136-149

Run every routed (token, slot) through its expert and add gate * output.

1. Zero output buffer; for each distinct expert id in `top_k_index` (ids outside `[0, E)` are skipped as 'no expert' sentinels): `token_idx, slot = where(top_k_index == e)`; compute the expert on those tokens; multiply by `top_k_weights[token_idx, slot]`; `index_add_` into the output.

Errors raised

Function	Exception	Message (f-string fields in braces)
`expert`	KeyError	Expert {expert_id} is neither retained nor replaced